

Contents

1	SOC : $\sqrt[3]{y}$	2
1.1	calculation of the cube root	2
1.2	C application code	6
1.3	Better Method, $\sqrt[3]{y}$	10
1.3.1	Source code file for cube root	10
1.3.2	Testbench code for the calculation of cube root	12
1.4	Zerocrossing	14
1.5	Testbench file	17
1.6	IP generation	19
1.7	Zerocrossing calculation (Better Method, Single FSM)	24
1.7.1	Source Code	24
1.7.2	Zerocrossing algorithm Testbench file	26
1.7.3	Simulations Result	27

Chapter 1

SOC : $\sqrt[3]{y}$

1.1 calculation of the cube root

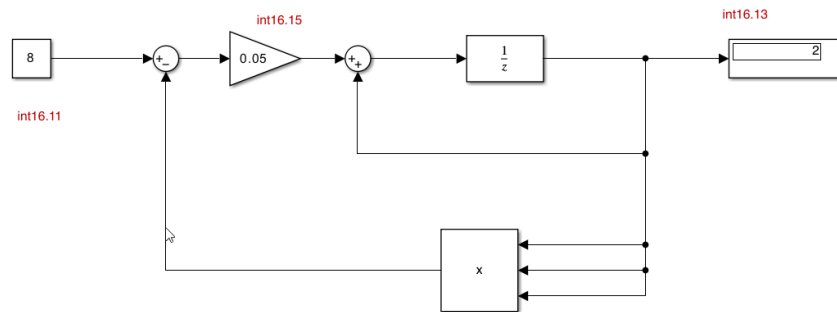


Figure 1.1: The system to calculate the cube of the input

The following code is the implementation of the above algorithm in vhdl:

```
1  -----
2  -- Engineer: Mohammad Mahdi Mohammadi
3  -- Create Date: 06/15/2026 11:07:11 AM
4  -- Module Name: cube - Behavioral
5  -- Description: accelerated cube root calculation
6  -- Revision: 2
7  -- Revision 0.01 - File Created
8  -----
9  LIBRARY IEEE;
10 USE IEEE.STD_LOGIC_1164.ALL;
11 USE IEEE.NUMERIC_STD.ALL;
12
13 ENTITY cube IS
14     PORT (
15         clk, resetn, start : IN STD_LOGIC;
16         done : OUT STD_LOGIC;
17         u : IN STD_LOGIC_VECTOR(15 DOWNTO 0);
18         y : OUT STD_LOGIC_VECTOR(15 DOWNTO 0));
19 END cube;
20
21 ARCHITECTURE Behavioral OF cube IS
22
23     --signal and data formate declaration for y^3 fsm
```

```

24  TYPE r3_state_type IS (idle, st1, st2, st3, st4);
25  SIGNAL r3_state : r3_state_type;
26
27  SUBTYPE int16_t IS INTEGER RANGE -32768 TO 32767;
28  SUBTYPE int32_t IS INTEGER RANGE -2147483648 TO 2147483647;
29
30  SIGNAL r3_in : int16_t := 16384;
31  SIGNAL r3_out : int16_t := 0;
32  SIGNAL r3_temp : int32_t;
33  SIGNAL r3_done : STD_LOGIC := '0';
34  SIGNAL r3_start : STD_LOGIC := '1';
35
36  --signal and datatype declaration for the main fsm
37  TYPE main_state_type IS (sidle, sst1, sst2, sst3, sst4, sst5, sst6);
38  SIGNAL main_state : main_state_type;
39  SIGNAL main_in, main_out : int16_t;
40  SIGNAL main_temp : int32_t;
41
42  SUBTYPE counter_type IS INTEGER RANGE 0 TO 25;
43  CONSTANT iteration : INTEGER := 25;
44  CONSTANT k01 : INTEGER := 1639; --k01 = 0.05 in int16.15
45  BEGIN
46
47  --fsm y^3
48  fsm_r3_p : PROCESS (clk, resetn)
49  BEGIN
50      IF rising_edge(clk) THEN
51          IF resetn = '0' THEN
52              r3_state <= idle;
53              r3_out <= 0;
54          ELSE
55              --default cases
56              r3_done <= '0';
57              CASE r3_state IS
58                  WHEN idle =>
59                      r3_done <= '1';
60                      r3_temp <= 0;
61                      IF r3_start = '1' THEN
62                          r3_state <= st1;
63                      END IF;
64                  WHEN st1 =>
65                      r3_temp <= r3_in * r3_in;
66                      r3_state <= st2;
67                  WHEN st2 =>
68                      r3_temp <= (r3_temp)/2 ** 13;
69                      r3_state <= st3;
70                  WHEN st3 =>
71                      r3_temp <= r3_temp * r3_in;
72                      r3_state <= st4;
73                  WHEN st4 =>
74                      r3_out <= r3_temp / (2 ** 15);
75                      r3_state <= idle;
76                  END CASE;
77          END IF;

```

```

78     END IF;
79 END PROCESS fsm_r3_p;
80 --fsm root cube calcuation
81 fsm_root_cube : PROCESS (clk, resetn)
82     VARIABLE cnt : counter_type := 0;
83 BEGIN
84     IF rising_edge(clk) THEN
85         IF resetn = '0' THEN
86             main_state <= sidle;
87             main_temp <= 0;
88             main_out <= 0;
89             cnt := 0;
90         ELSE
91             --default cases
92             done <= '0';
93             CASE main_state IS
94                 WHEN sidle =>
95                     done <= '1';
96                     cnt := 0;
97                     main_temp <= 0;
98                     main_in <= to_integer(signed(u));
99                     main_out <= 8192; --initializing the y(k) to 1.0
100                     ↪ in int16.13
101                     IF start = '1' THEN
102                         main_state <= sst1;
103                     END IF;
104                 WHEN sst1 =>
105                     IF cnt = iteration THEN
106                         main_state <= sst6;
107                     ELSE
108                         --calculating the y^3
109                         r3_in <= main_out;
110                         r3_start <= '1';
111                         main_state <= sst2;
112                     END IF;
113                 WHEN sst2 =>
114                     r3_start <= '0';
115                     IF r3_done = '1' THEN
116                         main_temp <= (main_in - r3_out); --int16.11
117                         main_state <= sst3;
118                     END IF;
119                 WHEN sst3 =>
120                     main_temp <= k01 * main_temp;
121                     main_state <= sst4;
122                 WHEN sst4 =>
123                     main_temp <= main_temp / (2 ** 13);
124                     main_state <= sst5;
125                 WHEN sst5 =>
126                     main_out <= main_out + main_temp;
127                     cnt := cnt + 1;
128                     main_state <= sst1;
129                 WHEN sst6 =>
130                     y <= STD_LOGIC_VECTOR(to_signed(main_out,
131                     ↪ y'length));

```

```

130         main_state <= sidle;
131     END CASE;
132 END IF;
133 END IF;
134 END PROCESS fsm_root_cube;
135 END Behavioral;

```

The following code shows the testbench for the above systems.

```

1  -----
2  -- Engineer: Mohammad Mahdi Mohammadi
3  -- Create Date: 06/15/2026 11:07:11 AM
4  -- Module Name: cube - Behavioral
5  -- Description: accelerated cube root calculation
6  -- Revision: 2
7  -- Revision 0.01 - File Created
8  -----
9  LIBRARY IEEE;
10 USE IEEE.STD_LOGIC_1164.ALL;
11 USE IEEE.NUMERIC_STD.ALL;
12 ENTITY cube_tb IS
13 END cube_tb;
14
15 ARCHITECTURE Behavioral OF cube_tb IS
16     COMPONENT cube IS
17     PORT (
18         clk, resetn, start : IN STD_LOGIC;
19         done : OUT STD_LOGIC;
20         u : IN STD_LOGIC_VECTOR(15 DOWNTO 0);
21         y : OUT STD_LOGIC_VECTOR(15 DOWNTO 0));
22     END COMPONENT cube;
23     SIGNAL clk, start, done : STD_LOGIC := '0';
24     SIGNAL resetn : STD_LOGIC := '1';
25     SIGNAL u : STD_LOGIC_VECTOR(15 DOWNTO 0) := x"0000";
26     SIGNAL y : STD_LOGIC_VECTOR(15 DOWNTO 0) := x"0000";
27
28     CONSTANT clk_period : TIME := 10 ns;
29
30 BEGIN
31
32     --clock generation
33     clk <= NOT clk AFTER clk_period/2;
34
35     --unit under test
36     uut : cube
37     PORT MAP(
38         clk => clk,
39         resetn => resetn,
40         start => start,
41         done => done,
42         u => u,
43         y => y);
44
45     --stimulus process

```

```

46 stim_p : PROCESS
47 BEGIN
48     start <= '1';
49     resetn <= '1';
50     u <= STD_LOGIC_VECTOR(to_signed(16384, 16));
51     WAIT FOR clk_period * 40;
52
53     WAIT;
54 END PROCESS stim_p;
55
56 END Behavioral;

```

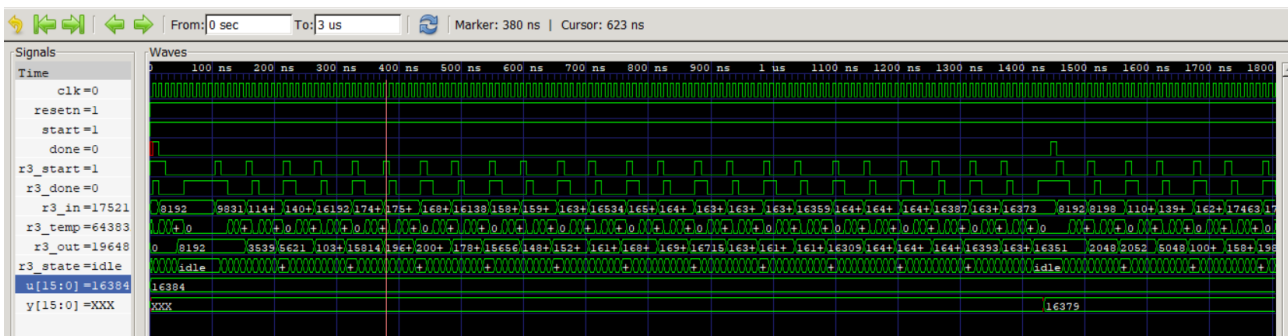


Figure 1.2: The system simulation using GHDL

1.2 C application code

The following code shows the c application built for this system.

```

1  /*
2   * helloworld.c: simple test application
3   *
4   * This application configures UART 16550 to baud rate 9600.
5   * PS7 UART (Zynq) is not initialized by this application, since
6   * bootrom/bsp configures it to baud rate 115200
7   *
8   * -----
9   * | UART TYPE   BAUD RATE                                |
10  * -----
11  *   uartns550   9600
12  *   uartlite    Configurable only in HW design
13  *   ps7_uart    115200 (configured by bootrom/bsp)
14  */
15
16  #include <stdio.h>
17  #include "platform.h"
18  #include "xil_printf.h"
19  #include <string.h>
20  #include <unistd.h>
21
22  #include "xparameters.h"
23

```

```

24 #define reg(k) *(volatile unsigned
    ↪ int*) (XPAR_CUBE_ROOT_0_S00_AXI_BASEADDR + 4*k)
25 #define cmd_len 128
26 #define nreg 4
27
28 void print_help(){
29     printf("x      : to exit the program \n");
30     printf("h      : to show this help menu \n");
31     printf("start : to start the calculation algorithm \n");
32     printf("test  : to test the peripherals \n");
33 }
34
35 void my_getline(char *cmd, int size){
36     int i = 0;
37     char c;
38     for(i = 0; i < size - 1; i++){
39         c = getchar();
40         if(c == '\r' || c == '\n'){
41             c = '\0';
42             putchar('\n');
43             putchar('\r');
44             break;
45         }
46         cmd[i] = c;
47         putchar(c);
48     }
49     cmd[i] = '\0';
50 }
51
52 int main()
53 {
54     int terminat = 0;
55     char cmd[cmd_len];
56     int xdata;
57
58     init_platform();
59     print_help();
60     do{
61         printf(">> "); fflush(stdout);
62         my_getline(cmd, cmd_len);
63         if(!strcmp(cmd, "x")){
64             terminat = 1;
65         } else if(!strcmp(cmd, "h")){
66             print_help();
67         } else if(!strcmp(cmd, "start")){
68             printf("enter the value to find the 3rd root \n");
69             my_getline(cmd, cmd_len);
70             double data0;
71             sscanf(cmd, "%lf", &data0);
72             if(data0 <= 0 || data0 > 10){
73                 printf("value out of range, please enter a value between
                    ↪ 0 and 10\n\r");
74                 continue; // go back to prompt, not break
75             }

```

```

76     xdata = (int)(data0 *(1 << 11));
77     reg(1) = xdata & 0xffff;
78
79     //triggering the start signal
80     reg(2) = 1;
81
82     //poling untill done is high
83     do{
84         xdata = reg(2);
85         usleep(10);
86     }while(!(xdata & 0x1));
87     short int y_raw = (short int)(reg(1) & 0xffff); //
88     ↪ sign-extend correctly
89     double data = (double)y_raw / (double)(1 << 13);
90
91     printf("the 3rd root of %lf is %lf \n", data0, data);
92 } else if(!strcmp(cmd, "test")){
93     printf("1. leds \n");
94     printf("2. push buttons \n");
95     printf("3. switches \n");
96
97     int choice;
98     printf("select one of the above options \n");
99     my_getline(cmd, cmd_len);
100     sscanf(cmd, "%d", &choice);
101
102     switch (choice) {
103     case 1:
104         printf("enter the value to be written to the leds \n");
105         my_getline(cmd, cmd_len);
106         sscanf(cmd, "%d", &xdata);
107         reg(0) = xdata;
108         break;
109
110     case 2:
111         printf("the push buttons will be read \n");
112         xdata = reg(0);
113         int pushbutton = (xdata >> 2) & 0xf;
114         printf("the value of the push buttons are : %x \n",
115             ↪ pushbutton);
116         break;
117
118     case 3:
119         printf("the push buttons will be read \n");
120         xdata = reg(0);
121         int sw = (xdata) & 0x3;
122         printf("the value of the switches are : %x \n", sw);
123         break;
124
125     default:
126         printf("you did not enter a valid input \n");
127         break;
128     }

```

```
128  
129     }  
130 }while(terminat == 0);  
131 print("Hello World\n\r");  
132 print("Successfully ran Hello World application");  
133 cleanup_platform();  
134 return 0;  
135 }
```

1.3 Better Method, $\sqrt[3]{y}$

The following code shows the implementation for the calculation of the cube root using one FSM method.

1.3.1 Source code file for cube root

The following code shows the source code implementation of the cube root calculation.

```
1  LIBRARY ieee;
2  USE ieee.std_logic_1164.ALL;
3  USE ieee.numeric_std.ALL;
4
5  ENTITY s25 IS
6      PORT (
7          clk : IN STD_LOGIC;
8          reset : IN STD_LOGIC;
9          start : IN STD_LOGIC;
10         done : OUT STD_LOGIC;
11         x_in : IN STD_LOGIC_VECTOR(15 DOWNTO 0);
12         y : OUT STD_LOGIC_VECTOR(15 DOWNTO 0)
13     );
14 END ENTITY s25;
15
16 ARCHITECTURE behavioral OF s25 IS
17     TYPE state_type IS (idle, init, state_cube0, state_cube1,
18         ↪ state_cube2, state_root0, state_root1);
19     SIGNAL state, next_state : state_type;
20
21     SUBTYPE int16_t IS INTEGER RANGE -32768 TO 32767;
22     SUBTYPE int32_t IS INTEGER RANGE -2147483648 TO 2147483647;
23     SUBTYPE counter_type IS INTEGER RANGE 0 TO 25;
24
25     CONSTANT input_frac : INTEGER := 11;
26     CONSTANT output_frac : INTEGER := 13;
27     CONSTANT iteration : INTEGER := 25;
28     CONSTANT k0 : INTEGER := 1639; -- 0.05 in (int16_15)
29
30     SIGNAL yk, yk1 : int16_t;
31     SIGNAL temp, temp_i : int32_t;
32
33     SIGNAL counter, counter_i : counter_type;
34 BEGIN
35     -- sequential process
36     seq_p : PROCESS (clk)
37     BEGIN
38         IF rising_edge(clk) THEN
39             IF reset = '1' THEN
40                 state <= idle;
41                 yk <= 0;
42                 temp <= 0;
43                 counter <= 0;
44             ELSE
```

```

45         state <= next_state;
46         yk <= yk1;
47         counter <= counter_i;
48         temp <= temp_i;
49     END IF;
50 END IF;
51 END PROCESS;
52
53 -- combinational process
54 comb_p : PROCESS (state, counter, start, yk, temp, x_in)
55 BEGIN
56     --default cases
57     next_state <= state;
58     done <= '0';
59     yk1 <= yk;
60     counter_i <= counter;
61     temp_i <= temp;
62
63     CASE state IS
64         WHEN idle =>
65             done <= '1';
66             IF start = '1' THEN
67                 next_state <= init;
68             END IF;
69         WHEN init =>
70             yk1 <= 0;
71             counter_i <= 0;
72             temp_i <= 0;
73             next_state <= state_cube0;
74         WHEN state_cube0 =>
75             temp_i <= yk * yk;
76             next_state <= state_cubel;
77         WHEN state_cubel =>
78             temp_i <= (temp / (2 ** 13)) * yk;
79             next_state <= state_cube2;
80         WHEN state_cube2 =>
81             temp_i <= temp / (2 ** 15);
82             next_state <= state_root0;
83         WHEN state_root0 =>
84             temp_i <= (to_integer(signed(x_in)) - temp) * k0;
85             next_state <= state_root1;
86         WHEN state_root1 =>
87             yk1 <= yk + temp/(2 ** 13);
88             IF counter < iteration THEN
89                 counter_i <= counter + 1;
90             END IF;
91
92             IF counter = iteration THEN
93                 next_state <= idle;
94             ELSE
95                 next_state <= state_cube0;
96             END IF;
97     END CASE;
98 END PROCESS;

```

```

99
100     y <= STD_LOGIC_VECTOR(to_signed(yk, y'length));
101
102 END ARCHITECTURE behavioral;

```

1.3.2 Testbench code for the calculation of cube root

The following code shows the testbench for the above implementation.

```

1  LIBRARY ieee;
2  USE ieee.std_logic_1164.ALL;
3  USE ieee.numeric_std.ALL;
4
5  ENTITY s25_tb IS
6  END ENTITY;
7
8  ARCHITECTURE behavioral OF s25_tb IS
9      COMPONENT s25 IS
10         PORT (
11             clk : IN STD_LOGIC;
12             reset : IN STD_LOGIC;
13             start : IN STD_LOGIC;
14             done : OUT STD_LOGIC;
15             x_in : IN STD_LOGIC_VECTOR(15 DOWNTO 0);
16             y : OUT STD_LOGIC_VECTOR(15 DOWNTO 0)
17         );
18     END COMPONENT s25;
19     SIGNAL clk : STD_LOGIC := '0';
20     SIGNAL reset, start, done : STD_LOGIC;
21     SIGNAL x_in : STD_LOGIC_VECTOR(15 DOWNTO 0);
22     SIGNAL y : STD_LOGIC_VECTOR(15 DOWNTO 0);
23
24     CONSTANT clk_period : TIME := 10 ns;
25 BEGIN
26
27     -- clock generation
28     clk <= NOT clk AFTER clk_period/2;
29
30     -- unit under test
31     uut : s25
32     PORT MAP(
33         clk => clk,
34         reset => reset,
35         start => start,
36         done => done,
37         x_in => x_in,
38         y => y
39     );
40
41     -- stimulus process
42     stim_p : PROCESS
43     BEGIN
44         start <= '1';

```

```

45     reset <= '0';
46     x_in <= STD_LOGIC_VECTOR(to_signed(16384, x_in'length));
47     WAIT FOR clk_period;
48     start <= '0';
49     WAIT FOR clk_period * 100;
50     WAIT;
51     END PROCESS;
52
53 END ARCHITECTURE behavioral;

```

The following figure shows the simulation result of the above implementation using GHDL. it shows the input value of 16384 which is equal to 8.0 in `int16_11` format and the output value of 8192 which is equal to 2.0 in `int16_13` format.

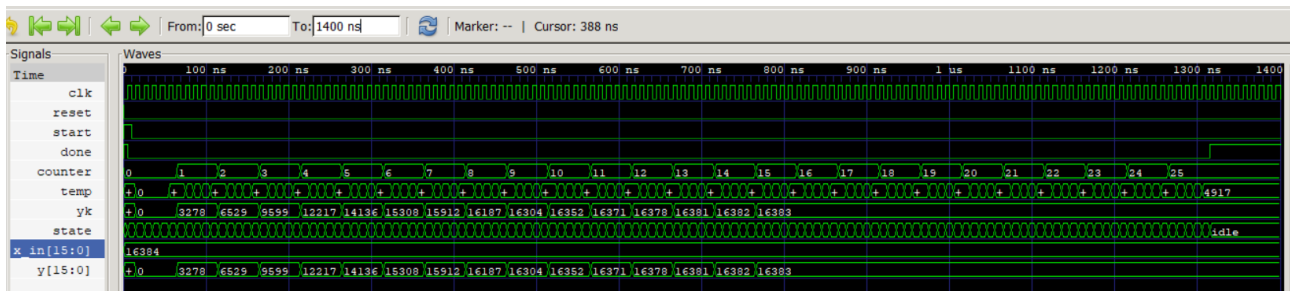


Figure 1.3: Simulation result of the cube root implementation

1.4 Zerocrossing

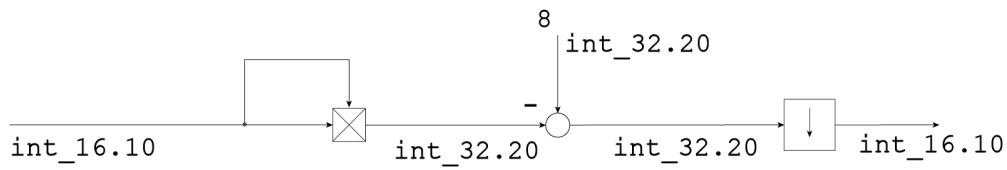


Figure 1.4: function calculation figure

The following vhdl code is the solution.

```

1  LIBRARY ieee;
2  USE ieee.std_logic_1164.ALL;
3  USE ieee.numeric_std.ALL;
4
5  ENTITY zerocross IS
6      PORT (
7          clk, resetn, start : IN STD_LOGIC;
8          done                : OUT STD_LOGIC;
9          rightv, leftv       : IN  STD_LOGIC_VECTOR(15 DOWNTO 0);
10         newx, newy           : OUT STD_LOGIC_VECTOR(15 DOWNTO 0)
11     );
12 END zerocross;
13
14 ARCHITECTURE behavioral OF zerocross IS
15
16     TYPE fcalc_state_type IS (idle, st1, st2, st3);
17     SIGNAL fcalc_state : fcalc_state_type := idle;
18
19     SUBTYPE int16_t IS INTEGER RANGE -32768 TO 32767;
20     SUBTYPE int32_t IS INTEGER RANGE -2147483648 TO 2147483647;
21
22     SIGNAL fcalc_xin    : int16_t := 0;
23     SIGNAL fcalc_yout   : int16_t := 0;
24     SIGNAL fcalc_temp   : int32_t := 0;
25     SIGNAL fcalc_done   : STD_LOGIC := '0';
26     SIGNAL fcalc_start  : STD_LOGIC := '0';
27
28     TYPE main_state_type IS (sidle, init, sst1, sst1b, sst2, sst3, sst4,
29         ↪ final);
30     SIGNAL main_state : main_state_type := sidle;
31
32     SUBTYPE counter_type IS INTEGER RANGE 0 TO 30;
33     SIGNAL count : counter_type := 0;
34
35     SIGNAL rightv_i, leftv_i : int16_t := 0;
36     SIGNAL newx_i, newy_i    : int16_t := 0;
37     CONSTANT iteration       : INTEGER := 15;
38
39 BEGIN
40     -- function FSM: f(x) = 8 - x^2
41     f_calc_process : PROCESS (clk)

```

```

42 BEGIN
43     IF rising_edge(clk) THEN
44         IF resetn = '0' THEN
45             fcalc_state <= idle;
46             fcalc_temp <= 0;
47             fcalc_yout <= 0;
48             fcalc_done <= '0';
49         ELSE
50             fcalc_done <= '0'; -- default
51
52             CASE fcalc_state IS
53                 WHEN idle =>
54                     -- no fcalc_done here
55                     IF fcalc_start = '1' THEN
56                         fcalc_state <= st1;
57                     END IF;
58
59                 WHEN st1 =>
60                     fcalc_temp <= fcalc_xin * fcalc_xin; -- Q20
61                     fcalc_state <= st2;
62
63                 WHEN st2 =>
64                     fcalc_temp <= 8388608 - fcalc_temp; -- 8.0 in
65                     -- Q20
66                     fcalc_state <= st3;
67
68                 WHEN st3 =>
69                     fcalc_yout <= fcalc_temp / (2**10); -- Q20 ->
70                     -- Q10
71                     fcalc_done <= '1'; -- only
72                     -- here
73                     fcalc_state <= idle;
74
75             END CASE;
76         END IF;
77     END IF;
78 END PROCESS f_calc_process;
79
80 -- bisection main FSM
81 zerocross_p : PROCESS (clk)
82     VARIABLE cnt : counter_type := 0;
83 BEGIN
84     IF rising_edge(clk) THEN
85         IF resetn = '0' THEN
86             main_state <= sidle;
87             newx_i <= 0;
88             newy_i <= 0;
89             rightv_i <= 0;
90             leftv_i <= 0;
91             count <= 0;
92             cnt := 0;
93         ELSE
94             done <= '0'; -- default
95             fcalc_start <= '0'; -- default

```

```

93
94     CASE main_state IS
95
96         WHEN sidle =>
97             done <= '1';
98             IF start = '1' THEN
99                 cnt := 0;
100                 count <= 0;
101                 main_state <= init;
102             END IF;
103
104         WHEN init =>
105             rightv_i <= to_integer(signed(righttv));
106             leftv_i <= to_integer(signed(lefttv));
107             main_state <= sst1;
108
109         WHEN sst1 =>
110             IF cnt = iteration THEN
111                 main_state <= final;
112             ELSE
113                 -- compute midpoint, wait one cycle for
114                 -- ↪ signal to settle
115                 newx_i <= (rightv_i + leftv_i) / 2;
116                 main_state <= sst1b;
117             END IF;
118
119         WHEN sst1b =>
120             -- newx_i is now valid, feed to function block
121             fcalc_xin <= newx_i;
122             fcalc_start <= '1';
123             main_state <= sst2;
124
125         WHEN sst2 =>
126             -- wait for function FSM to complete
127             IF fcalc_done = '1' THEN
128                 -- fcalc_yout valid on next cycle
129                 main_state <= sst3;
130             END IF;
131
132         WHEN sst3 =>
133             -- fcalc_yout is now stable
134             newy_i <= fcalc_yout;
135             main_state <= sst4;
136
137         WHEN sst4 =>
138             IF newy_i < 0 THEN
139                 rightv_i <= newx_i;
140             ELSE
141                 leftv_i <= newx_i;
142             END IF;
143             cnt := cnt + 1;
144             count <= cnt;
145             main_state <= sst1;

```

```

146         WHEN final =>
147             newx      <= STD_LOGIC_VECTOR(to_signed(newx_i,
148             ↪ newx'length));
149             newy      <= STD_LOGIC_VECTOR(to_signed(newy_i,
150             ↪ newy'length));
151             done       <= '1';
152             main_state <= sidle;
153
154         END CASE;
155     END IF;
156 END PROCESS zerocross_p;
157 END ARCHITECTURE behavioral;

```

1.5 Testbench file

The following code is the testbench file of the above program

```

1  LIBRARY ieee;
2  USE ieee.std_logic_1164.ALL;
3  USE ieee.numeric_std.ALL;
4
5  ENTITY zerocross_tb IS
6  END zerocross_tb;
7
8  ARCHITECTURE behavioral OF zerocross_tb IS
9      COMPONENT zerocross IS
10         PORT (
11             clk, resetn, start : IN STD_LOGIC;
12             done : OUT STD_LOGIC;
13             rightv, leftv : IN STD_LOGIC_VECTOR(15 DOWNTO 0);
14             newx, newy : OUT STD_LOGIC_VECTOR(15 DOWNTO 0)
15         );
16     END COMPONENT zerocross;
17     SIGNAL clk, start : STD_LOGIC := '0';
18     SIGNAL resetn : STD_LOGIC := '1';
19     SIGNAL done : STD_LOGIC;
20     SIGNAL rightv, leftv : STD_LOGIC_VECTOR(15 DOWNTO 0) := (OTHERS =>
21     ↪ '0');
22     SIGNAL newx, newy : STD_LOGIC_VECTOR(15 DOWNTO 0) := (OTHERS =>
23     ↪ '0');
24
25     --signals for the clock generation
26     CONSTANT clk_period : TIME := 10 ns;
27
28 BEGIN
29     --clock generation
30     clk <= NOT clk AFTER clk_period/2;
31
32     --unit under test
33     uut : zerocross

```

```

32  PORT MAP(
33      clk => clk,
34      resetn => resetn,
35      start => start,
36      done => done,
37      rightv => rightv,
38      leftv => leftv,
39      newx => newx,
40      newy => newy
41  );
42
43  --stimulus process
44  stim_p : PROCESS
45  BEGIN
46      rightv <= STD_LOGIC_VECTOR(to_signed(4096, 16));
47      leftv <= STD_LOGIC_VECTOR(to_signed(-4096, 16));
48
49      -- reset pulse
50      resetn <= '0';
51      WAIT FOR clk_period * 2;
52      resetn <= '1';
53      WAIT FOR clk_period;
54
55      start <= '1';
56      resetn <= '1';
57      wait for clk_period;
58      start <= '0';
59      WAIT FOR clk_period * 200;
60
61
62      WAIT;
63  END PROCESS stim_p;
64  END behavioral;

```

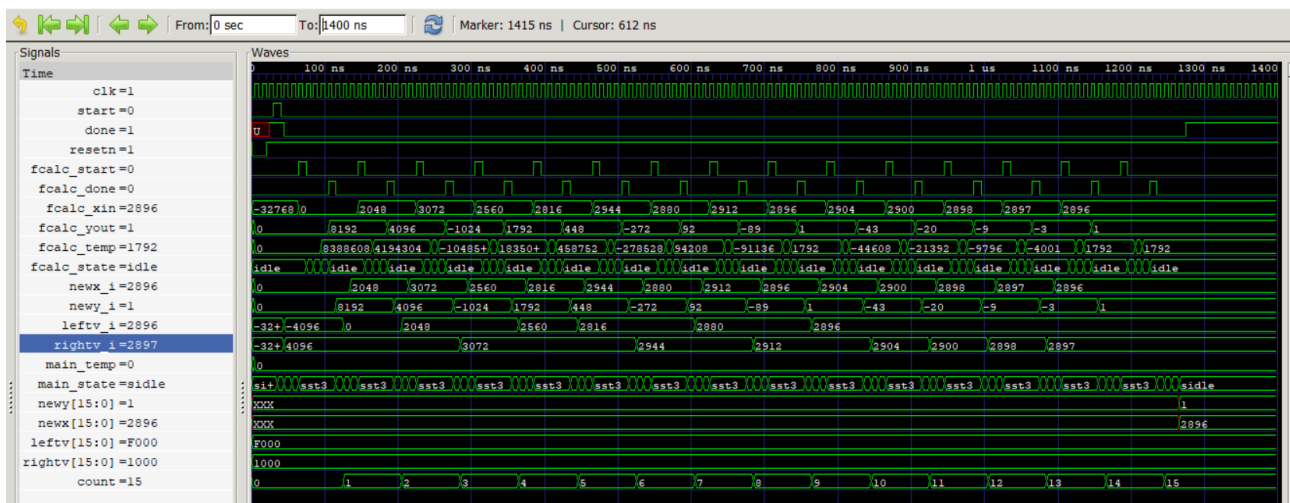


Figure 1.5: output of zero-crossing detector

1.6 IP generation

The following code shows the vhd code for reading and writing the registers.

```
1  -- Implement memory mapped register select and write logic generation
2  -- The write data is accepted and written to memory mapped registers
   ↳ when
3  -- axi_awready, S_AXI_WVALID, axi_wready and S_AXI_WVALID are
   ↳ asserted. Write strobes are used to
4  -- select byte enables of slave registers while writing.
5  -- These registers are cleared when reset (active low) is applied.
6  -- Slave register write enable is asserted when valid address and
   ↳ data are available
7  -- and the slave is ready to accept the write address and write
   ↳ data.
8  slv_reg_wren <= axi_wready and S_AXI_WVALID and axi_awready and
   ↳ S_AXI_AWVALID ;
9
10 process (S_AXI_ACLK)
11 variable loc_addr : std_logic_vector(OPT_MEM_ADDR_BITS downto 0);
12 begin
13     if rising_edge(S_AXI_ACLK) then
14         if S_AXI_ARESETN = '0' then
15             slv_reg0 <= (others => '0');
16             slv_reg1 <= (others => '0');
17             slv_reg2 <= (others => '0');
18             slv_reg3 <= (others => '0');
19             start <= '0';
20         else
21             start <= '0';
22             loc_addr := axi_awaddr(ADDR_LSB + OPT_MEM_ADDR_BITS downto
   ↳ ADDR_LSB);
23             if (slv_reg_wren = '1') then
24                 case loc_addr is
25                     when b"00" =>
26                         for byte_index in 0 to (C_S_AXI_DATA_WIDTH/8-1) loop
27                             if ( S_AXI_WSTRB(byte_index) = '1' ) then
28                                 -- Respective byte enables are asserted as per write
   ↳ strobes
29                                 -- slave register 0
30                                 slv_reg0(byte_index*8+7 downto byte_index*8) <=
   ↳ S_AXI_WDATA(byte_index*8+7 downto byte_index*8);
31                             end if;
32                         end loop;
33                     when b"01" =>
34                         for byte_index in 0 to (C_S_AXI_DATA_WIDTH/8-1) loop
35                             if ( S_AXI_WSTRB(byte_index) = '1' ) then
36                                 -- Respective byte enables are asserted as per write
   ↳ strobes
37                                 -- slave register 1
38                                 slv_reg1(byte_index*8+7 downto byte_index*8) <=
   ↳ S_AXI_WDATA(byte_index*8+7 downto byte_index*8);
39                             end if;
40                         end loop;
```

```

41         when b"10" =>
42             start <= '1';
43             for byte_index in 0 to (C_S_AXI_DATA_WIDTH/8-1) loop
44                 if ( S_AXI_WSTRB(byte_index) = '1' ) then
45                     -- Respective byte enables are asserted as per write
46                     ↪ strobes
47                     -- slave register 2
48                     slv_reg2(byte_index*8+7 downto byte_index*8) <=
49                         ↪ S_AXI_WDATA(byte_index*8+7 downto byte_index*8);
50                 end if;
51             end loop;
52         when b"11" =>
53             for byte_index in 0 to (C_S_AXI_DATA_WIDTH/8-1) loop
54                 if ( S_AXI_WSTRB(byte_index) = '1' ) then
55                     -- Respective byte enables are asserted as per write
56                     ↪ strobes
57                     -- slave register 3
58                     slv_reg3(byte_index*8+7 downto byte_index*8) <=
59                         ↪ S_AXI_WDATA(byte_index*8+7 downto byte_index*8);
60                 end if;
61             end loop;
62         when others =>
63             slv_reg0 <= slv_reg0;
64             slv_reg1 <= slv_reg1;
65             slv_reg2 <= slv_reg2;
66             slv_reg3 <= slv_reg3;
67         end case;
68     end if;
69 end if;
70 end process;
71
72 -- Implement memory mapped register select and read logic generation
73 -- Slave register read enable is asserted when valid address is
74 ↪ available
75 -- and the slave is ready to accept the read address.
76 slv_reg_rden <= axi_arready and S_AXI_ARVALID and (not
77 ↪ axi_rvalid) ;
78
79 process (leftv, rightv, newx, done, newy, axi_araddr,
80 ↪ S_AXI_ARESETN, slv_reg_rden)
81 variable loc_addr :std_logic_vector(OPT_MEM_ADDR_BITS downto 0);
82 begin
83     -- Address decoding for reading registers
84     loc_addr := axi_araddr(ADDR_LSB + OPT_MEM_ADDR_BITS downto
85 ↪ ADDR_LSB);
86     case loc_addr is
87         when b"00" =>
88             reg_data_out <= x"0000" & leftv;
89         when b"01" =>
90             reg_data_out <= x"0000" & rightv;
91         when b"10" =>
92             reg_data_out <= x"000" & "000" & newx & done;

```

```

87         when b"11" =>
88             reg_data_out <= x"0000" & newy;
89         when others =>
90             reg_data_out <= (others => '0');
91     end case;
92 end process;
93
94 leftv <= slv_reg0(15 downto 0);
95 rightv <= slv_reg1(15 downto 0);

```

The following code shows the c application for the above program

```

1  /*
2  * helloworld.c: simple test application
3  *
4  * This application configures UART 16550 to baud rate 9600.
5  * PS7 UART (Zynq) is not initialized by this application, since
6  * bootrom/bsp configures it to baud rate 115200
7  *
8  * -----
9  * | UART TYPE   BAUD RATE                                |
10 * -----
11 *   uartns550   9600
12 *   uartlite    Configurable only in HW design
13 *   ps7_uart    115200 (configured by bootrom/bsp)
14 */
15
16 #include <stdio.h>
17 #include "platform.h"
18 #include "xil_printf.h"
19 #include <string.h>
20 #include <unistd.h>
21
22 #include "xparameters.h"
23
24 #define reg(k) *(volatile unsigned int *) (XPAR_ZCROSS_0_S00_AXI_BASEADDR
↵ + (4*k))
25 #define cmd_len 64
26
27 void print_help(){
28     printf("h      : to show this help menu \n");
29     printf("x      : to close the application \n");
30     printf("start : to start the calculation \n");
31 }
32
33 void my_getline(char *cmd, int size){
34     int i;
35     char c;
36     for(i = 0; i < size - 1; i++){
37         c = getchar();
38         if(c == '\r' || c == '\n'){
39             c = '\0';
40             putchar('\r');
41             putchar('\n');

```

```

42         break;
43     }
44     cmd[i] = c;
45     putchar(c);
46 }
47 cmd[i] = '\0';
48 }
49
50 int main()
51 {
52     int terminat = 0;
53     char cmd[cmd_len];
54     unsigned int xdata;
55     init_platform();
56
57     print_help();
58     do{
59         printf(">> "); fflush(stdout);
60         my_getline(cmd, cmd_len);
61         if(!strcmp(cmd, "x")){
62             terminat = 1;
63         } else if(!strcmp(cmd, "write", 5)){
64             unsigned int regnum;
65             unsigned int regval;
66             if(sscanf(cmd, "write %d %d", &regnum, &regval) != 2){
67                 printf("the value was wrote correctly, right usage is
68                 ↪ write <regnum> <regval> \n");
69             } else {
70                 reg(regnum) = regval;
71                 printf("the value %d has been written to reg%d. \n",
72                 ↪ regval, regnum);
73             }
74         } else if(!strcmp(cmd, "start")){
75             double rightv_f, leftv_f;
76             unsigned int rightv, leftv;
77             printf("please enter the value for rightv and leftv.
78             ↪ (int_16.10), <rightv> <leftv> \n");
79             my_getline(cmd, cmd_len);
80             if(sscanf(cmd, "%lf %lf", &rightv_f, &leftv_f) != 2){
81                 printf("you did not enter the values correctly \n");
82             } else {
83                 rightv = (unsigned int)(rightv_f * (1 << 10));
84                 leftv = (unsigned int)(leftv_f * (1 << 10));
85
86                 reg(0) = leftv;
87                 reg(1) = rightv;
88                 printf("%d : has be written to reg(0) \n", leftv);
89                 printf("%d : has be written to reg(1) \n", rightv);
90
91                 //triggering the start signal
92                 reg(2) = 1;
93
94                 //polling for done signal
95                 do{

```

```

93         xdata = reg(2);
94         usleep(10);
95     }while(!(xdata & 0x1));
96     printf("done flage raised \n");
97
98     //xdata = reg(2);
99     printf("xdata = reg(2) = %d \n", xdata);
100
101     short unsigned int newx, newy;
102     double newx_f, newy_f;
103     xdata = (xdata >> 1);
104     newx = *(short unsigned int*)&(xdata);
105     newx_f = (double)newx / (double)(1 << 10);
106
107     xdata = reg(3);
108     printf("xdata = %d \n", xdata);
109     newy = *(short unsigned int*)&(xdata);
110     newy_f = (double)newy / (double)(1 << 10);
111
112     printf("the zerocrossing value for this program has been
113     ↪ found \n");
114     printf("the zerocrossing is : %d (%lf) \n", newx,
115     ↪ newx_f);
116     printf("the correspodng y value is : %d (%lf) \n",
117     ↪ newy, newy_f);
118 }
119 }
120 }while(terminat == 0);
121
122 print("Thanks for using the zcross application\n\r");
123 print("Successfully ran application");
124 cleanup_platform();
125 return 0;
126 }

```

1.7 Zerocrossing calculation (Better Method, Single FSM)

The zerocrossing method has been implemented using a single FSM method. The following code shows the implementation of the zerocrossing calculation using a single FSM method.

1.7.1 Source Code

The following code shows the source code for the zerocrossing calculation using a single FSM method.

```
1  LIBRARY ieee;
2  USE ieee.std_logic_1164.ALL;
3  USE ieee.numeric_std.ALL;
4
5  ENTITY s24 IS
6      PORT (
7          clk : IN STD_LOGIC;
8          reset : IN STD_LOGIC;
9          start : IN STD_LOGIC;
10         done : OUT STD_LOGIC;
11         xin : IN STD_LOGIC_VECTOR(31 DOWNTO 0);
12         yout : OUT STD_LOGIC_VECTOR(15 DOWNTO 0)
13     );
14 END ENTITY s24;
15
16 ARCHITECTURE behavioral OF s24 IS
17     TYPE state_type IS (idle, init, state1, state2, state3, state4,
18         ↪ state5);
19     SIGNAL state, next_state : state_type;
20     SUBTYPE int16_t IS INTEGER RANGE -32768 TO 32767;
21     SUBTYPE int32_t IS INTEGER RANGE -2147483648 TO 2147483647;
22     SUBTYPE counter_type IS INTEGER RANGE 0 TO 15;
23
24     CONSTANT rightv_init : int16_t := 4096; -- 4 in int16_10
25     CONSTANT leftv_init : int16_t := 1024; -- 1 in int16_10
26
27     SIGNAL counter, counter_i : counter_type;
28     SIGNAL temp, temp_i : int32_t;
29     SIGNAL newx : int16_t := 0;
30     SIGNAL leftv, leftv_i, rightv, rightv_i : int16_t;
31 BEGIN
32
33     -- sequential process
34     seq_p : PROCESS (clk)
35     BEGIN
36         IF rising_edge(clk) THEN
37             IF reset = '1' THEN
38                 state <= idle;
39                 counter <= 0;
40                 temp <= 0;
41                 rightv <= 0;
42                 leftv <= 0;
43             ELSE
```

```

44         state <= next_state;
45         counter <= counter_i;
46         temp <= temp_i;
47         leftv <= leftv_i;
48         rightv <= rightv_i;
49     END IF;
50 END IF;
51 END PROCESS seq_p;
52
53 -- combinational process
54 comb_p : PROCESS (state, start, counter)
55 BEGIN
56     -- default cases
57     done <= '0';
58     next_state <= state;
59     counter_i <= counter;
60     temp_i <= temp;
61     leftv_i <= leftv;
62     rightv_i <= rightv;
63
64     CASE state IS
65         WHEN idle =>
66             done <= '1';
67             IF start = '1' THEN
68                 next_state <= init;
69             END IF;
70         WHEN init =>
71             rightv_i <= rightv_init;
72             leftv_i <= leftv_init;
73             counter_i <= 0;
74             newx <= 0;
75             next_state <= state1;
76         WHEN state1 =>
77             newx <= (leftv + rightv)/2;
78             next_state <= state2;
79         WHEN state2 =>
80             temp_i <= (newx * newx);
81             next_state <= state3;
82         WHEN state3 =>
83             temp_i <= (temp - to_integer(signed(xin)));
84             next_state <= state4;
85         WHEN state4 =>
86             temp_i <= temp / (2 ** 13);
87             IF counter <= 15 THEN
88                 counter_i <= counter + 1;
89             END IF;
90             next_state <= state5;
91         WHEN state5 =>
92             IF temp >= 0 THEN
93                 rightv_i <= newx;
94             ELSE
95                 leftv_i <= newx;
96             END IF;
97             IF counter = 15 THEN

```

```

98         next_state <= idle;
99     ELSE
100         next_state <= state1;
101     END IF;
102 END CASE;
103 END PROCESS comb_p;
104
105 yout <= STD_LOGIC_VECTOR(to_signed(newx, yout'length));
106
107 END ARCHITECTURE behavioral;

```

1.7.2 Zerocrossing algorithm Testbench file

The following code shows the testbench for the above implementation of the zerocrossing calculation using a single FSM method.

```

1  LIBRARY ieee;
2  USE ieee.std_logic_1164.ALL;
3  USE ieee.numeric_std.ALL;
4
5  ENTITY s24_tb IS
6  END ENTITY;
7
8  ARCHITECTURE behavioral OF s24_tb IS
9      COMPONENT s24 IS
10         PORT (
11             clk : IN STD_LOGIC;
12             reset : IN STD_LOGIC;
13             start : IN STD_LOGIC;
14             done : OUT STD_LOGIC;
15             xin : IN STD_LOGIC_VECTOR(31 DOWNTO 0);
16             yout : OUT STD_LOGIC_VECTOR(15 DOWNTO 0)
17         );
18     END COMPONENT;
19     SIGNAL clk : STD_LOGIC := '0';
20     SIGNAL reset, start, done : STD_LOGIC;
21     SIGNAL xin : STD_LOGIC_VECTOR(31 DOWNTO 0);
22     SIGNAL yout : STD_LOGIC_VECTOR(15 DOWNTO 0);
23
24     CONSTANT clk_period : TIME := 10 ns;
25
26 BEGIN
27
28     -- clock generation
29     clk <= NOT clk AFTER clk_period/2;
30
31     -- unit under test
32     uut : s24
33     PORT MAP(
34         clk => clk,
35         reset => reset,
36         start => start,
37         done => done,

```

```

38     xin => xin,
39     yout => yout
40 );
41
42 -- stimulus process
43 stim_p : PROCESS
44 BEGIN
45     reset <= '0';
46     start <= '1';
47     xin <= STD_LOGIC_VECTOR(to_signed(8388608, xin'length));
48     WAIT FOR clk_period;
49     start <= '0';
50     WAIT FOR clk_period * 50;
51
52     WAIT;
53 END PROCESS;
54
55 END ARCHITECTURE behavioral;

```

1.7.3 Simulations Result

The following figure shows the simulation result of the above implementation using GHDL. It shows the input value of 8388608 which is equal to 8.0 in `int16_10` format and the output value of 2894 which is equal to 2.8262 in `int16_10` format.

